Problem A. Ah, It's Yesterday Once More

This is a construction task. The random program repeatedly chooses one of the four directions uniformly, so the intended counterexample is not a complicated maze but a tree-shaped grid on which random movement is unlikely to merge all kangaroos within the allowed number of moves.

The useful quantity is the effective diameter of the empty-cell tree. A spiral wastes too many cells on walls; it creates long paths, but the success probability of the random sequence is still too high. A better construction keeps almost all used cells in one tree and repeatedly bends the path upward-left and downward-left, creating a long, thin structure with large distances between some leaves. Since the empty cells form a connected acyclic graph, the input is legal, and the long bottleneck makes the randomized solution fail with probability at least the required threshold.

It is enough to hard-code one verified board. During implementation, run an independent checker for connectedness, acyclicity, and the empirical failure probability of the randomized program.

Problem B. Baby's First Suffix Array Problem

Build the suffix array, the rank array, and an RMQ structure on the LCP array of the original string. For a query (l, r, k), the required rank equals

$$1 + \#\{x \in [l, r] \mid s[x..r] < s[k..r]\}.$$

The comparison is not always the same as comparing the infinite suffixes $s[x..n]$ and $s[k..n]$, because one of the compared strings may end first at position r.

Let $y = \mathrm{rank}(k)$ and let $\mathrm{lcp}(u, v)$ be obtained by RMQ on the suffix-array interval. First count all suffix-array positions before y whose starting position lies in $[l, r]$. This gives the number of x with $s[x..n] < s[k..n]$. Then correct the pairs whose ordering is changed by truncation at r:

- for $x < k$, subtract those with $\mathrm{lcp}(x, k) \geq r - k + 1$, because then $s[k..r]$ is a prefix of $s[x..r]$;
- for $x > k$, add those with $\mathrm{lcp}(x, k) \geq r - x + 1$, because then $s[x..r]$ is a prefix and becomes smaller.

The three counted terms are offline two-dimensional dominance queries on suffix-array rank and original position, with an LCP-threshold condition. They can be processed by divide-and-conquer on the suffix array, Fenwick trees, and the minimum LCP values crossing the middle.

Complexity. $O((n + m) \log^2 n)$ or better, depending on the offline counting implementation.

Problem C. Certain Scientific Railgun

The final choice can be described by an axis-aligned rectangle. Every robot must lie strictly above, below, left, or right of the rectangle, and the cost is the perimeter of the rectangle minus the Manhattan distance saved at one chosen corner. Thus the task becomes optimizing such a rectangle under point-side constraints.

Enumerate the lower boundary of the rectangle. While this boundary is moved upward, points that can no longer be placed below impose new restrictions on the upper boundary and on the minimum horizontal span. For every candidate upper boundary, maintain the best possible value of the left-right contribution. When a point invalidates an upper boundary, delete that candidate from an ordered set. The remaining candidates give the best rectangle for the current lower boundary.

The main implementation detail is to update the horizontal gap contribution lazily while boundaries are inserted or erased. With coordinate compression, the sweep only changes at robot coordinates.

Complexity. $O(n \log n)$ after sorting and compression.

Problem D. Degree of Spanning Tree

Start from any spanning tree T. Since the sum of degrees in a tree is $2(n-1)$, at most one vertex can have degree greater than $n/2$. If no such vertex exists, T is already valid.

Suppose u is the unique bad vertex. Remove one tree edge incident to u; this splits the tree into a component containing u and another component C. If there is a non-tree edge from C to a vertex outside C that is not u, replacing the removed edge by that edge decreases the degree of u and preserves connectivity. Repeat this exchange until u is no longer bad. If a second bad vertex appears during the process, the two bad vertices must be adjacent, and the exchange is performed with this exceptional edge treated separately.

A randomized Kruskal variant with the constraint $\deg(v) \le n/2$ also works in practice, but the exchange argument gives a deterministic proof of existence and termination.

Complexity. $O(nm)$ with straightforward searches; faster implementations are possible.

Problem E. Evil Coordinate

Only the number of moves in each of the four directions matters if we decide to execute all equal directions consecutively. Try every permutation of the four direction blocks, expand it into a path, and test whether the forbidden coordinate is ever visited.

For a fixed order, the path consists of at most four axis-parallel segments. A segment hits the forbidden point if the point lies on the same horizontal or vertical line and its coordinate lies between the two endpoints of the segment. If one of the 24 orders avoids the forbidden point, output the corresponding string. Otherwise the answer is impossible.

Complexity. $O(n)$ per test case, since the number of tested orders is constant.

Problem F. Fireworks

Assume Kotori makes exactly k fireworks before trying to launch. One attempt costs $nk + m$ time, and the probability that at least one of the k fireworks works is

$$1 - (1-p)^k.$$

Therefore the expected time is

$$E(k) = \frac{nk + m}{1 - (1-p)^k}.$$

The function is unimodal for positive integer k, so the minimum can be found by ternary search on integers, or by locating the real minimum and checking nearby integers. The answer is $\min_{k \ge 1} E(k)$.

Complexity. $O(\log U)$ evaluations with ternary/binary search on a sufficiently large upper bound U.

Problem G. Go

Treat white stones as vertices of a four-neighbor graph and maintain for each connected component the number of adjacent empty cells, called liberties. The query result is determined only by components adjacent to the changed cell, except when deleting a white stone disconnects its component.

If the flipped cell is black, the affected white components are the four-neighbor components around it. After the flip, update their liberty counts and count the components whose liberties become zero.

If the flipped cell is white, the component may split. This can only create new components when the cell is an articulation point in the white graph. Build the block-cut tree of the white graph. For an articulation point, the incident biconnected components describe exactly the pieces that remain after removal; their liberty counts decide which pieces become dead. Non-articulation points are handled directly.

Complexity. $O(nm)$ preprocessing for the grid graph and $O(1)$ or degree-proportional work per local query, depending on representation.

Problem H. Harmonious Rectangle

Count the complement: colorings with no harmonious rectangle. The answer is then 3^{nm} minus this number.

If one dimension is 1, no rectangle exists, so the answer is 0. For larger dimensions, the avoidance problem is tiny despite the original bound. By the pigeonhole principle and a small exhaustive check, if the larger dimension exceeds 8 and the smaller dimension is at least 2, every coloring contains a harmonious rectangle. Thus the complement is zero in that case.

For the remaining cases both dimensions are at most 8. Use DFS over rows, representing a row as a base-3 mask. When appending a new row, compare it with every previous row; the pair of rows is forbidden if there are two columns whose two vertical pairs are equal in the required way. Precompute compatibility between row masks and perform DP over valid prefixes.

Complexity. Constant-time precomputation for all small shapes; each test case is answered in $O(1)$ after symmetry $n \le m$.

Problem I. Interested in Skiing

There are only $2n$ relevant endpoints. For every ordered pair of endpoints, test whether one can ski from the first to the second without violating the geometry. If yes, add a directed edge whose weight is the slope or difficulty of that segment.

The original question becomes a minimum bottleneck path problem on this directed graph: find a path whose maximum edge weight is as small as possible. Binary search the answer and run DFS/BFS using only edges with weight at most the current threshold.

Complexity. $O(n^3 + n^2 \log n)$ with the direct graph construction and binary search.

Problem J. Just Another Game of Stones

For a normal Nim position with xor sum x, a winning first move chooses a pile y and changes it to $x \oplus y$, which is legal exactly when $y > x \oplus y$. This condition depends only on the highest set bit of x: the pile y is valid iff it has a 1 in that bit.

Thus a query on piles $b_l, \ldots, b_r$ plus the extra pile x needs the xor sum of all piles and, if the xor is nonzero, the number of piles whose value has the highest set bit of the xor. Maintain for every segment its xor and the count of numbers containing each bit.

The update is range $b_i \leftarrow \max(b_i, x)$. This is handled by Segment Tree Beats. For each node, keep the minimum value, second minimum value, and the count of the minimum. If x lies strictly between the

minimum and second minimum, only all minimum elements are raised to x; update the xor and the bit counts using the parity/count of those minimum elements. Otherwise recurse.

Complexity. $O((n+q)\log n \log A)$ amortized, with $A < 2^{30}$.

Problem K. K Co-prime Permutation

If $k = 0$, no solution exists because position 1 always satisfies $\gcd(1, p_1) = 1$.

For $k \geq 1$, rotate the first k values and leave the rest fixed:

$$p_1 = k, \qquad p_i = i - 1 \ (2 \leq i \leq k), \qquad p_i = i \ (i > k).$$

Then the first k positions all have coprime pairs: $\gcd(1, k) = 1$ and $\gcd(i, i-1) = 1$. Every remaining position $i > k$ has $p_i = i$, so $\gcd(i, p_i) = i > 1$. Therefore exactly k positions are coprime.

Complexity. $O(n)$.

Problem L. Let's Play Curling

For a red stone to score, there must be a center c that is closer to this red stone than to every blue stone. A group of red stones can be scored together exactly when their positions lie inside one maximal interval of the line that contains no blue stone position; a blue stone position blocks the interval.

Sort and compress all red and blue positions. Mark positions occupied by blue stones. Scan the compressed line and sum the multiplicities of red stones in each consecutive segment of positions not occupied by blue. The maximum segment sum is the best score. If every segment sum is zero, print "Impossible".

Complexity. $O((n+m)\log(n+m))$ per test case.

Problem M. Monster Hunter

Root the tree at vertex 1. Use a tree knapsack DP. Let

$$dp[u][j][0/1]$$

be the minimum cost needed to eliminate all vertices in the subtree of u, using j magic deletions there; the last flag records whether u itself is deleted by magic. When u is eliminated normally, its cost includes its own value and the values of children that have not already been magically removed.

Merge the children one by one. If a child v is magically deleted, the transition subtracts the contribution w_v that would otherwise be paid when deleting u. This is the only non-standard part of the knapsack. After all children are merged, initialize the two cases for deleting or not deleting u and propagate the minimum values upward.

The answers for using exactly $0, 1, \ldots, n$ magic operations are the corresponding minima at the root.

Complexity. $O(n^2)$ memory and time with the usual subtree-size optimization.